

A Two-Layer Ensemble Architecture for Enhanced Directional Price Prediction in Financial Markets

Sahand Hassanizorgabad

Koc University, School of Sciences and Engineering, Sarıyer, 34450, Istanbul, Turkey

Semih Yumusak

University of Southampton, School of Electronics and Computer Science, Southampton, SO17 1BJ, Hampshire, United Kingdom

Abstract

Financial markets exhibit nonlinear dynamics and complexity, where diverse assets are traded between buyers and sellers, each seeking to maximise their return on investment (ROI). Predicting short-term price movements is particularly challenging due to the interaction of multiple unpredictable factors, including stock-specific news, company profiles, public sentiment, and macroeconomic conditions. Existing approaches often struggle with evaluation bias and lack the flexibility to operate across different financial markets. This paper presents a novel two-layer Composing Ensembles architecture, consisting of an ensemble of models optimized through grid search and a self-improving meta-learner layer, to enhance daily price directional prediction for financial instruments. The proposed system is back-tested on a diverse range of assets, demonstrating a 2.76% improvement in predictive accuracy and outperforming a benchmark strategy in terms of risk-adjusted returns and consistency across market conditions.

Keywords: Algorithmic Trading, Stock Prediction, Financial Forecasting, Machine Learning, Ensemble Learning, Meta Learning, Backtesting

1. Introduction

A financial market is a nonlinear complex system in which various traders and investors seek to maximize returns on investment through the trading of diverse assets [1]. As demonstrated by Lo [2], these markets exhibit properties of complex adaptive systems, with fractal patterns and chaotic behavior emerging from the collective actions of market participants. Forecasting financial markets presents an ongoing challenge, as these markets are continuously evolving and subject to high volatility, driven by factors such as company-specific news, macroeconomic events, public sentiment, and international economic conditions. These interdependent variables create a dynamic environment, making it difficult to achieve reliable predictions of market trends [3]. Over the years, numerous forecasting methods (from traditional statistical models to more contemporary machine learning techniques) have been developed, yet consistently reliable predictions remain elusive due to the inherent stochasticity of financial data. More important for traders, investors, and financial institutions than perfect accuracy in predicting day-to-day price fluctuations is the ability to achieve even a small improvement in the direction of those predictions, as even marginal gains can yield substantial financial benefits.

Since the early 2000s, one of the most significant advancements in this context has been the development of algorithmic trading [3]. As documented by Hendershott et al. [4], this approach has fundamentally transformed market microstructure by automating the decision-making process using computer algorithms that follow predefined criteria. By operating at speeds and frequencies beyond human capability, these systems can leverage vast amounts of historical and real-time data to forecast short-term price movements. Kirilenko and Lo [5] demonstrate how algorithmic trading has revolutionized market dynamics, while Cartea et al. [6] provide comprehensive evidence of how these systems enable the execution of complex strategies in real time strategies that would be impractical for human traders to implement manually. By exploiting statistical arbitrage opportunities, price discrepancies, and temporal market inefficiencies, these systems allow for faster and more precise trading decisions, contributing to their widespread adoption in the financial sector.

Despite the significant advancements in algorithmic trading, challenges remain, particularly in achieving high accuracy and consistent profitability over time. A major limitation of many existing models is their focus on a single financial instrument or a narrow set of assets, which makes them inflexible and unsuitable for adapting to different market conditions. Bailey et al. [7] demonstrate how this narrow focus often leads to pseudo-mathematical overfitting, while Lopez de Prado [8] shows that traditional backtesting methods can give false confidence in model performance. While these models may perform well in certain market phases, they often struggle to maintain performance when applied to different instruments or under changing conditions. Bailey et al. [7] demonstrate how this narrow focus often leads to pseudo-mathematical overfitting, while Lopez de Prado [8] shows that traditional backtesting methods can give false confidence in model performance. Furthermore, while higher accuracy may be an important metric, it does not always translate into higher return on investment (ROI), particularly when models are overfitted[9] to specific data sets or market scenarios. The inability to dynamically adapt to changing market conditions remains a significant weakness in traditional algorithmic trading models, limiting their long-term effectiveness.

Building on recent advances in ensemble methods [10] and meta-learning approaches [11], this paper seeks to address these limitations by proposing a flexible, multi-model, multi-instrument trading system designed to improve both prediction accuracy and ROI. Unlike traditional models that focus on a single instrument or a limited range of assets, Two-Layer Composing Ensembles architecture allows users to select a list of instruments and dynamically adjust both models and instruments during trading. This approach extends the statistical arbitrage framework proposed by Krauss et al. [12] by incorporating adaptive meta-learning components [13]. This dynamic adaptability is achieved through the first layer, which consists of an ensemble of models optimized via grid search. Each model is trained on the corresponding input data for every selected financial instrument, ensuring tailored predictions. The second layer, a self-improving meta-learner, observes the performance of the first layer and adjusts the filtering mechanism to fine-tune strategy performance, making the system highly adaptable to various market conditions. This flexibility ensures that the system can adapt to different market phases, avoiding downturns in specific instruments and enhancing its overall performance. The model also offers the ability for users to choose instruments based on their preferences or regulatory requirements, making the system more versatile and accessible. Following Lopez de Prado’s methodology [14], we evaluate model performance using a range of metrics[15], including precision[16], recall[17], F1-score, and backtesting results [18], which provides

a more comprehensive view of performance and ensures that accuracy improvements correlate with improved financial outcomes. Furthermore, the system is designed to be scalable and adaptable, allowing for the continuous addition of new models and datasets, which will improve performance over time. In summary, this paper introduces a novel, flexible, and highly customizable algorithmic trading system that improves both accuracy and ROI, while also being adaptable to evolving market conditions and regulatory environments.

2. Related Works

Numerous studies in the academic literature have focused on predicting individual financial instruments or a limited subset of assets, employing various machine learning [19, 20], deep learning [21], and statistical approaches [22, 23, 24]. While these studies provide valuable insights into financial forecasting, they often lack the flexibility required for broader market applications. In contrast, this research adopts a multi-instrument and multi-model framework, offering greater adaptability in asset selection, improved predictive reliability, and the potential for higher returns on investment. Given that the proposed approach is based on an ensemble of predictive strategies, where multiple models contribute to forecasting the next candlestick’s direction—the following subsections will present a detailed review of the literature on candlestick-based strategies and ensemble learning methodologies.

2.1. Candlestick Algorithms

Nearly all algorithms utilize raw candlestick data, which consists of open, close, high, and low prices of a particular time frame. For example, the work of [25] investigates the detection of candlestick patterns by machine learning models and analyzes how such patterns can be used as a trading strategy to predicting the future market. In contrast, the work by [26] do not focus on pattern detection but instead utilize 26 features derived from raw candlestick data, including data from the previous three days, to predict the next candlestick movement.

One of the most relevant studies, [21], uses candlestick data with conjunction of osilators as our work, however they only choose bitcoin data with comparing multiple strategies. despite of having nearly same input to our algorithms, the output of this paper is more promising since it is tested on multiple instruments in a different time periods that market had a different conditions. paper [21] presents GRU model as their best model however their evaluation is based on precision, since each candlesticks return is different metrics such as accuracy, percision can’t be reliable for example in their work their third best model with LSTM has a 58.41% precision however this model average return is -0.011% , This discrepancy highlights Evaluation Bias and this is because some wrong predicted candles had more return and this is long candles are regular in bitcoin chart.

In another study, the authors of paper [24] propose a multivariate financial time-series stock forecasting model that integrates sequential pattern mining and sequence similarity. This approach enhances stock trend prediction accuracy by applying K-line pattern mining to multivariate time-series data. Briefly, each data point is labeled and categorized into candlestick patterns, as described in [27]. The primary objective of paper [24] is to predict the direction of the next candlestick, with their proposed hybrid model (Sub-KP + Sim-KP) achieving an average accuracy of 56.05% and 55.56% across two datasets.

However, Evaluation Bias may also affect this study, as the reliance on accuracy as the primary evaluation metric does not fully capture the profitability of trading decisions. Additionally, the study is limited to only two datasets, which restricts its generalizability across diverse market conditions. In contrast, this paper proposes that such algorithms can be further refined and effectively incorporated into ensemble methods. Specifically, integrating robust models such as those presented in [24],[25],[25] and [21] within an ensemble meta-learner framework could potentially yield improved predictive performance and enhanced financial returns.

2.2. Ensemble Algorithms

Ensemble methods are a class of learning algorithms that construct a set of classifiers and classify new data points by taking a (weighted) vote of their predictions [28]. Generally, these methods are more reliable than any single strategy due to the flexibility they allow for the algorithm to determine the weight of each strategy in different market conditions. In addition, [10] reports that, generally, error rates of ensemble committees are among the lowest among other approaches. For example, in [29], the author benchmarks various strategies against an ensemble of thresholded convolutional neural networks for candlestick pattern detection. The results prove that the ensembles of TCNNs outperform other strategies. Similarly, in [11], three classification methods outperformed traditional benchmarks with significant alpha coefficients for higher cumulative returns over a long trading period. Moreover, [30] presents positive results regarding the performance of the proposed multiple DC-strategy GA compared to nine benchmarks, namely five single DC-based trading strategies, three technical analysis indicators, and the buy-and-hold strategy, widely used as a financial benchmark.

3. Methodological Approach

Trading algorithms live and die by their ability to process massive amounts of financial data into profitable decisions. Every algorithmic trading system contains the same building blocks: data fetching, preprocessing, core model development, backtesting, and results presentation [31]. We introduce a two-layer Composing Ensembles approach: one with the addition of a self-improving meta-learner ensemble layer on top of the first layer. This meta-learner further enhances the benefits of the ensemble approach by continuously refining its performance at each instant of time. The meta-learner trains iteratively to improve overall system performance, with which the system is able to outperform others even by using conventional models in the first layer.

Figure 1 illustrates the data flow of the algorithm, from fetching financial data to detecting potential profitable instrument-model pairs.

3.1. Data Representation and Description

The time series data for this project is sourced from the yfinance library and OANDA broker APIs, encompassing price data for supported symbols. Each dataset, as shown in Table 1, comprises six quantitative features: Open, High, Low, Close, Adj Close, and Volume [27].

During each training cycle, a list of financial instruments is fetched from the data provider. Based on the method used from the instrument-model search part of the algorithm, generally, more instruments will yield better results concerning the ROI [32]. On the other hand, this also requires increased computational power, which consequently slows down the training phase.

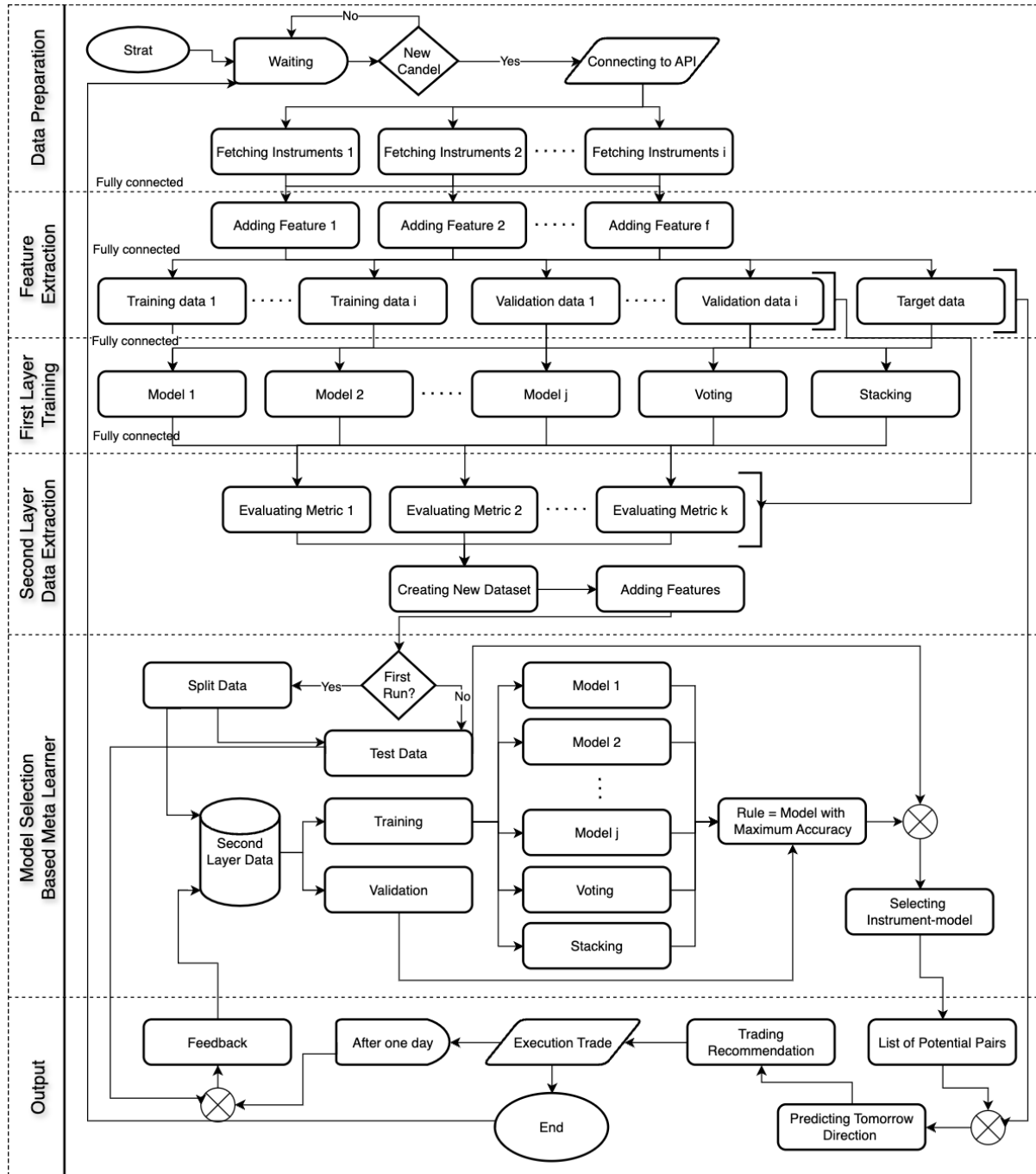


Figure 1: Swimlane Diagram

Table 1: Input Data

Date	Open	High	Low	Close	Adj Close	Volume
2013-12-24	1199.80	1205.60	1197.70	1205.10	1205.10	184
2013-12-26	1207.10	1215.90	1207.10	1214.10	1214.10	140
2013-12-27	1213.40	1218.50	121.90	1216.10	1216.10	278
2013-12-30	1215.00	1215.00	1194.40	1203.10	1203.10	351
...	...	...	...	...	...	...

3.2. Pre-processing

During preprocessing, the raw data is processed to become more usable for model training. New features are generated and added to the dataset to improve the prediction accuracy. Afterward, all datasets are shifted by adding the features of previous days to each data point, forming a time-series structure. The summary statistics of the final input data for the first-layer models is illustrated in Table 3.

3.2.1. Feature Extraction

New features are generated for each instrument, including return, label, and indicators. Table 2 demonstrates all added features and their corresponding calculation formulas.

Table 2: Features Formulas

Feature	Formula
Return	$\frac{\text{Close} - \text{Open}}{\text{Open}} \times 100$
SMA[33]	$\frac{\text{Sum of Prices}}{\text{Number of Periods}}$
RSI[34]	$100 - \left(\frac{100}{1 + \text{RS}} \right)$
MACD[35] Line	$\text{EMA}_{12} - \text{EMA}_{26}$
Signal Line	9-day EMA of MACD Line
Histogram	MACD Line – Signal Line

3.2.2. Features Importance with SHAP Values

SHAP (SHapley Additive exPlanations) [36] values are based on Shapley values from cooperative game theory, adapted for feature attribution in machine learning models. They quantify the contribution of each feature to a specific prediction by treating the prediction process as a game and computing the marginal contribution of a feature across all possible feature subsets. The formula for SHAP values is:

$$\varphi_i = \sum_{S \subseteq \mathcal{F} \setminus \{i\}} \frac{|\mathcal{S}|!(|\mathcal{F}| - |\mathcal{S}| - 1)!}{|\mathcal{F}|!} [f(\mathcal{S} \cup \{i\}) - f(\mathcal{S})]$$

Where:

- φ_i is the SHAP value for feature i .
- $\mathcal{F}$ is the set of all features.
- $\mathcal{S}$ is a subset of $\mathcal{F}$ that does not include i .
- $f(\mathcal{S})$ is the model's prediction when only the features in subset $\mathcal{S}$ are present.
- $|\mathcal{S}|$ is the size of subset $\mathcal{S}$.
- $(|\mathcal{F}| - |\mathcal{S}| - 1)!$ is the factorial of the remaining features excluding i .

- $|\mathcal{F}|$ is the factorial of the total number of features (normalization factor).

The formula computes the marginal contribution of feature i by evaluating how much adding it to subset S changes the model output. Contributions are averaged over all possible subsets S , weighted by the number of permutations of features in S and not in S .

The SHAP value outputs, presented in Figures 2 and 3 in the Evaluation and Results section, illustrate the rationale behind incorporating shifted previous days as inputs for first layer and Normalized Accuracy feature as input for the second layer.

3.2.3. Shifting Previous-Days

After adding extra features, each data point is shifted according to its previous point, creating a time series data structure similar to that created by the TimeseriesGenerator function, which will be used for Keras preprocessing. Using previous-day features makes a huge difference in time series model training as pointed in [37]. Thus, the feature with the highest SHAP value in their dataset is P1DR (Previous-Day Return), ranking 1st with a value of 0.723. From our work and the analysis of SHAP values for each model, we also found that previous-day features have a substantial impact on the results. The features can be return, volume, or close prices in the input data while training the first layer.

3.2.4. Partitioning Steps

After data collection and preprocessing steps, the extracted features are presented in Table 3. This table refers to the Gold (GC=F) instrument, while the rest of the instruments have an identical set of features. In the following steps, the datasets will be used for model development and evaluation. Since this is a short-term prediction algorithm, the data will be divided into training and test sets in a 95-5% ratio, where 95% will be used for training. The remaining 5% will be kept aside strictly for the final evaluation of the model. Further, the training set will again be divided into learning and validation sets in a 90-10% ratio for better fine-tuning and validation of the models. However, for real-time implementation, a test set is not required.

Table 3: Summary statistics of first-layer model inputs before partitioning

	Close	Volume	Return	RSI	EMAF	EMAM	EMAS
Count	2515	2515	2515	2500	2496	2416	2366
Mean	1593.3	5295.6	0.000074	52.1	1590.8	1581.0	1576.8
Std	385.0	29205.8	0.007990	11.9	378.7	354.4	342.0
Min	1050.8	0.000000	-0.05302	21.0	1071.4	1103.2	1117.5
25%	1257.7	46.0	-0.0035	44.3	1258.6	1258.8	1263.3
50%	1508.0	173.0	0.000000	51.7	1506.9	1517.3	1521.6
75%	1865.2	540.0	0.0040	60.4	1863.7	1850.2	1831.5
Max	2788.5	386334.0	0.052884	87.0	2713.3	2575.8	2508.5

3.3. Composing Ensembles of Instruments-Models

The system structure consists of a learning phase, validation evaluation, and the detection of successful models among all trained models for all instruments. Let's formalize this as a function composition: Potential Pairs = $(ML \circ E \circ M)(x)$ where M is the first learning layer, E is the evaluation of M , and ML is the second layer, which selects the

best instrument-model pairs based on evaluation metrics of the models. Here, x represents the input, a list of instruments along with their associated time series data. The final prediction will be the selected instrument and its pre-trained model. The input to the Meta Learner will be a 3-dimensional matrix, this tensor will have dimensions $i * j * k$, where i represents the number of instruments, j represents the number of models, and k corresponds to the evaluation metrics.

$$\text{Potential Pairs} = \text{Meta Learner} \left(\left[E_1 \ E_2 \ \dots \ E_k \right] \otimes \left(\begin{bmatrix} M_1 \\ M_2 \\ \vdots \\ M_j \end{bmatrix} \otimes \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_i \end{bmatrix} \in \mathbb{R}^{mp \times nq} \right) \in \mathbb{R}^{m \times n} \right)$$

3.3.1. Training Classification Models with Instruments

In the learning phase, each of the classification models listed in Table 4, including their ensembles [28], voting classifier [38] and stacking classifier [39], will be trained with a list of datasets belonging to different instruments. The result will be a number of instrument-model pairs equal to the product of the number of models and the number of instruments. The training models are hyperparameter-tuned using the GridSearch method to enhance their overall accuracy.

Table 4: List of Machine-Learning Models

Model Type	Hyperparameters
Gradient Boosting [40]	$n_{\text{estimators}} = 100$, learning rate = 0.01, max depth = 12
Logistic Regression [41]	$C = 0.1$, max iter = 10000, solver = 'lbfgs'
DecisionTreeClassifier [42]	max depth = 12, min samples split = 6, min samples leaf = 4
RandomForestClassifier [43]	$n_{\text{estimators}} = 500$, max depth = 10
KNeighborsClassifier [44]	$n_{\text{neighbors}} = 7$, weights = 'distance'
GaussianNB [45]	var smoothing = $1e - 09$
LinearSVC [46]	$C = 1$, max iter = 10000
MLPClassifier [47]	hidden layer sizes = 69, max iter = 10000, activation = 'relu', alpha = 0.0001
SupportVectorClassifier [48]	$C = 1$, kernel = 'rbf', gamma = 'scale', max iter = 5000

3.3.2. Preparing Second Layer Input with Evaluating Pairs

Following the learning phase, the models undergo validation with dedicated datasets, where their performance is assessed using various metrics such as accuracy, backtest, nnp, NormalizedAcc, precision, recall, F1-score, and AUC. These metrics play a crucial role in evaluating the effectiveness of the models and guiding the training of new machine-learning model to identify a successful classification model and its instrument to be utilized.

All metrics are sourced from sklearn.metrics [49], except for backtest, nnp, and NormalizedAcc. The backtest metric represents the output of a function that displays the return of the validation dataset using the considered model. Nnp signifies the normal return profit without any machine learning assistance. NormalizedAcc represents normalized accuracy calculated using the following formula, Here, counts[0] represents the

number of Label 1 samples, which corresponds to a green candle or bullish day, and counts[0] represents the number of red candles or bearish days:

$$f(x) = \begin{cases} 1 - \frac{\text{Number of Label 1 Samples}}{\text{Total Number of Samples}} & \text{Counter}[1] < \text{Counter}[0] \\ \frac{\text{Number of Label 1 Samples}}{\text{Total Number of Samples}} & \text{Counter}[1] \geq \text{Counter}[0] \end{cases}$$

$$\text{NormalizedAcc} = f(x)(\text{Accuracy} * 0.5)$$

Certainly, considering the potential issue with models setting all outputs to 0 or 1 to achieve higher accuracy in the presence of imbalanced labels, the formula for NormalizedAcc can be adapted to address this challenge. NormalizedAcc provides a more balanced evaluation, considering the baseline accuracy that could be achieved by random guessing. And we can observe in Figure 3 that the SHAP values indicate NormalizedAcc has the highest importance among the features.

As a result of the initial phase of training, a data frame is generated that illustrates the effectiveness of models for each symbol. In this study, the results are shown for 16 symbols and 11 models, producing a total of 176 rows.

- D_i represent the i -th dataset where $i = 1, 2, \dots, 16$.
- M_j represent the j -th model where $j = 1, 2, \dots, 11$.
- k represents the number of evaluation metrics (e.g., accuracy, precision, recall, F1-score).
- E_{ijk} represent the evaluation metric of k for model M_j on dataset D_i .

$$\text{SecondLayerInput} = \begin{bmatrix} D_1 & M_1 & E_{111} & E_{112} & \cdots & E_{11k} \\ D_1 & M_2 & E_{121} & E_{122} & \cdots & E_{12k} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ D_i & M_j & E_{ij1} & E_{ij2} & \cdots & E_{ijk} \end{bmatrix}$$

The evaluations are stored in a data frame where the rows or indexes correspond to the model-dataset pairs and the columns correspond to the evaluation metrics. An example of the first layer output and second layer input can be visualized in Table 5, which, once partitioned, is ready to be fed into the second layer model. This model identifies profitable instruments along with their pre-trained machine learning models.

Table 5: Input of Second-Layer training

Dataset	Model	Accuracy	NormalizeAcc	...	Profit %	Label
AAPL	LogisticRegression	0.519078	0.492486	...	-0.7686	0
AAPL	DecisionTreeClassifier	0.504680	0.478824	...	-0.8717	0
⋮	⋮	⋮	⋮	⋮	⋮	⋮
GC=F	SVC	0.532084	0.504098	...	-0.5121	0
GC=F	MLPClassifier	0.488825	0.463115	...	-0.4681	0

3.3.3. Detecting Profitable Pairs with Meta Learner

This layer will select the profitable pairs of instruments and models from the historical performance data and continuously get better with more data, which in turn increases the accuracy of the whole system. In this layer, multiple classification models-as in the

first layer-are trained on historical performance data. This data is kept in the database from previous training sessions of the system. The meta-learner then predicts the label for inputs of new training sessions and gives feedback regarding its prediction on the following day. That means, it just works like reinforcement mechanisms, where the system keeps improving after each training.

This meta-learner is rule-based, selecting the model with the highest accuracy on the validation data. It is mentioned earlier that the validation and training data for this layer are sourced from the database. Figure 1 will help understand this step more clearly. The overall accuracy of the system is very much dependent upon the predictions to be made by this second layer of prediction. If this layer correctly predicts the profitable instrument and model, then the subsequent operations will go ahead. Consequently, this layer achieves an average accuracy of around 80% at present.

4. Evaluations and Results

In this section, we present a comprehensive analysis of our model’s performance through multiple evaluation approaches, examining feature contributions using SHAP (SHapley Additive exPlanations) values at different model layers and validating the strategy through historical backtesting.

4.1. SHAP Values of first Layer models

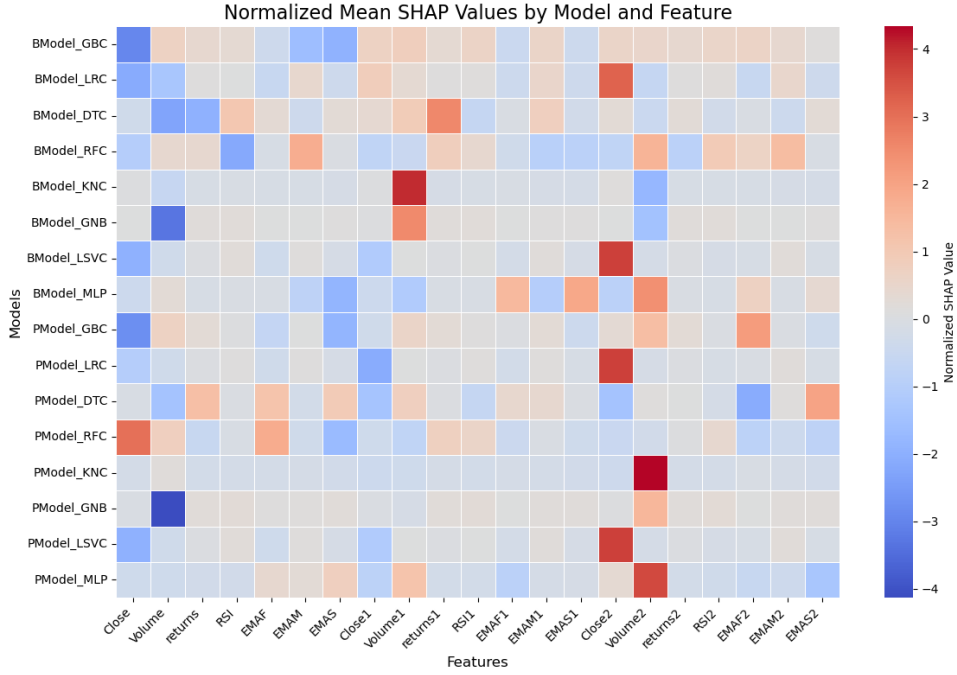


Figure 2: Heatmap of SHAP Values by Models and Features

Figure 2 shows the feature importance analysis, shedding light on some variables that play an important role in the training process. Most of the models considered the closing price and volume of trade for two days ago as highly important features. Additionally, for some models, first and second close prices and volumes are also of importance. This suggests that time-shifted values of the data bear a lot of significance for prediction accuracy.

4.2. SHAP Values of Second Layer Model

Figure 3 shows the feature importance of used inputs in the second layer. SHAP values are the mean contribution of all models of the second layer. Normalized model accuracy and profit percentage are the most critical features to predict the model performance in this context.

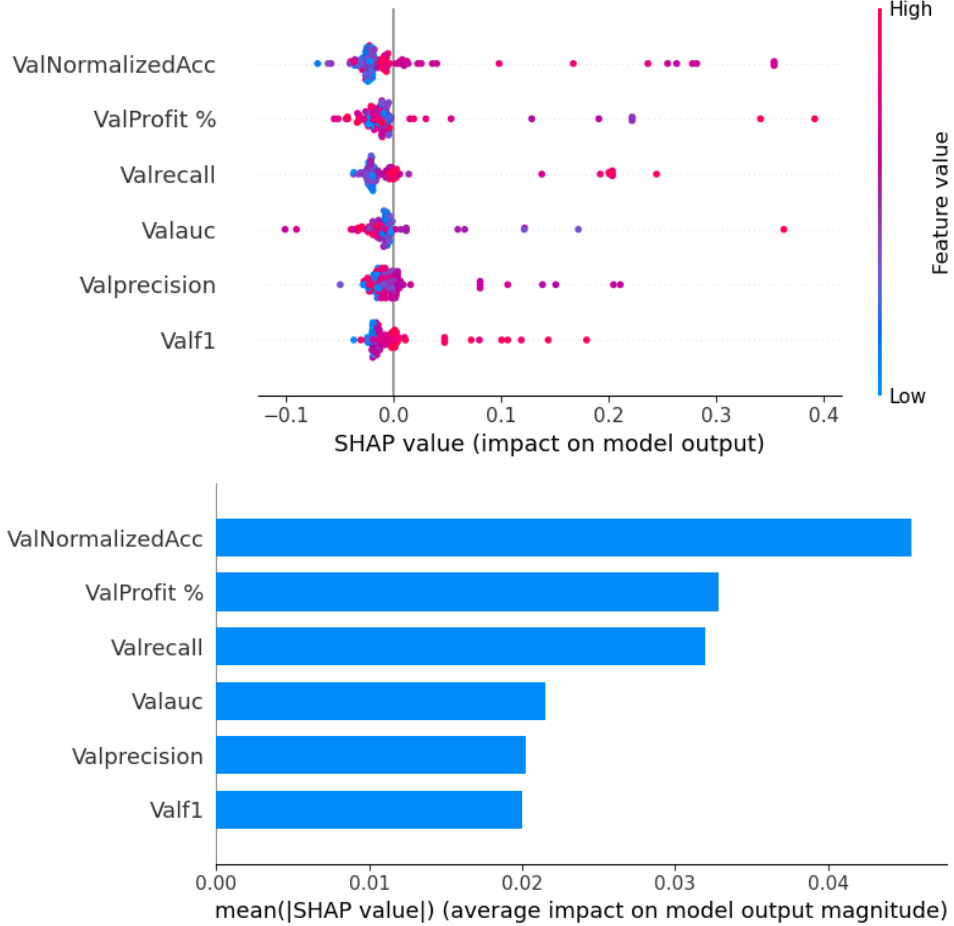


Figure 3: SHAP Values of Second Layer model

4.3. Backtest Results

To obtain the project's results, first, the instruments in Table 6 are employed.

Among all the selected symbol-model pairs, those chosen by the best-model-finder are going to be tested on the test dataset. The best-model-finder employs two selection methods: one for choosing the best symbol-model, and the other for selecting lists of symbol-model pairs predicted to be profitable. This method is set manually. In Figure 4, four subcharts exist, each trained at different times. This adjustment is made by selecting the best symbol-model for each specific time period. The "StrategyReturns," depicted by the green line, represents the return of selected symbols enhanced by their chosen machine learning models. On the other hand, the "NormalReturns," illustrated by the blue line, signifies the return of selected symbols without employing any machine learning techniques.

In Figure 5, there are four subcharts, each trained at different times with various instruments. This adjustment is made by selecting lists of symbol-models for each specific

Table 6: List of Assets Used as Inputs

Ticker Symbol	Financial Instruments, Assets
AAPL	Apple Inc.
GOOGL	Alphabet Inc. (Google)
AMZN	Amazon.com Inc.
TSLA	Tesla, Inc.
META	Meta Platforms, Inc. (formerly Facebook)
CSCO	Cisco Systems, Inc.
NVDA	NVIDIA Corporation
NFLX	Netflix Inc.
JPM	JPM - JPMorgan Chase & Co.
CN50	China 50 Index
IBM	International Business Machines Corporation
Corn-USD	Corn
BTC-USD	Bitcoin to US Dollar
ETH-USD	Ethereum to US Dollar
BCO-USD	Brent Crude Oil
GC=F	Gold



Figure 4: Results of single instrument (best pair)

time period. The “ModelsReturn”, depicted by the blue line, represents the return of selected symbols enhanced by their chosen machine learning models. On the other hand, the “NormalReturn,” illustrated by the green line, signifies the return of selected symbols without employing any machine learning techniques.



Figure 5: Result of chosen instrument-model pairs

Figure 6 and Figure 7 are selected to show the ROI of the method presented in paper [20], implemented over both long-term and short-term periods. By this, it may be concluded that although the mean accuracy of the approach in this paper is lower than that of paper [24], the ROI is much higher. The best part is that all this can be done in a very short period, say one month, as it allows great flexibility regarding the choice of instruments and strategies.

4.4. Algorithmic Efficiency

The study in [30] was executed on a non-dedicated Red Hat Enterprise Linux system with 24 cores and 24 GB of memory, where the training process took 330 minutes to complete. In contrast, our system, executed on an Apple Silicon ARM64 with an M2 architecture, completed training in 80 minutes.

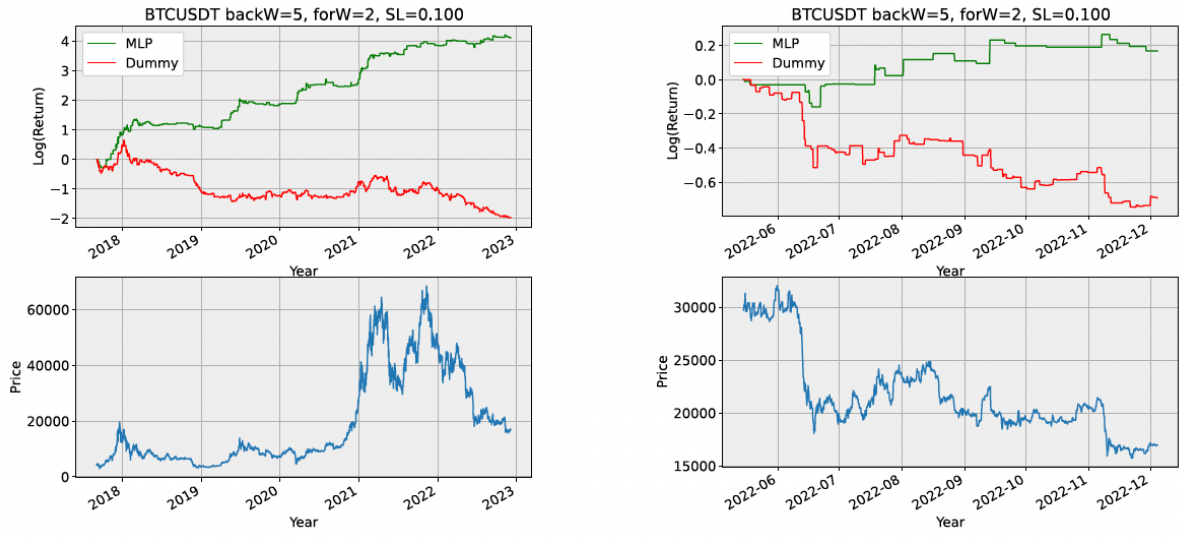


Figure 6: BTC-USD Logarithmic return

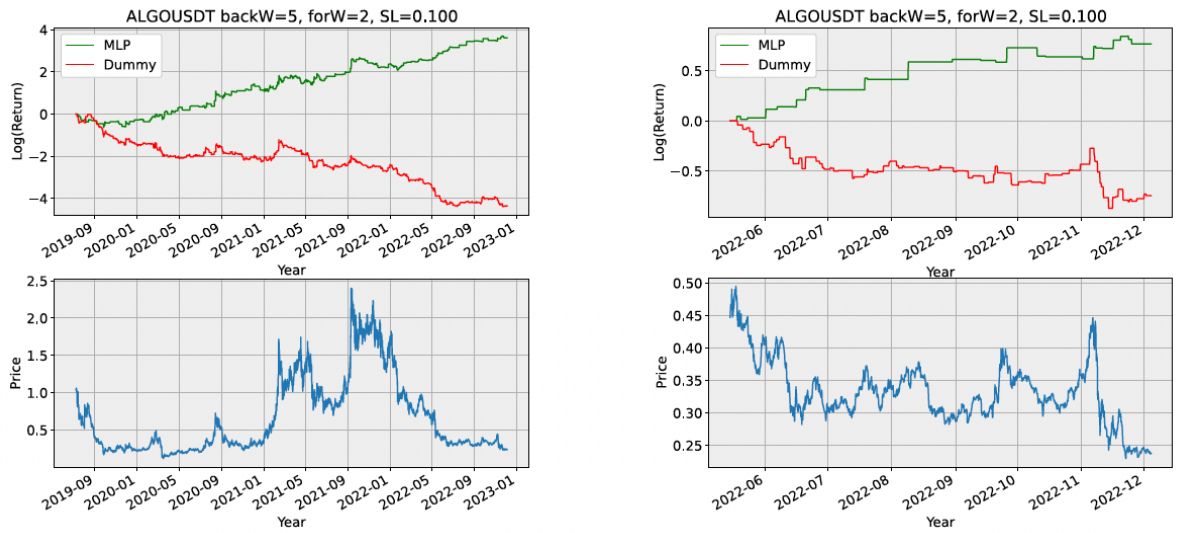


Figure 7: ALGO-USD Logarithmic return

Table 7: Algorithm evaluation after each training session

Model Selector	Model Selector ACC	Sample Size	Mean ROI	Mean Benchmark ROI	Mean Improvement	ACC
VOT	0.87	403	0.002868	0.001638	0.001231	53.24
RFC	0.82	806	0.028473	0.002197	0.026276	55.29
STK	0.84	1209	0.029538	0.003503	0.026035	56.86
VOT	0.86	1612	0.030498	-0.002967	0.033465	57.14
VOT	0.87	2015	0.034690	0.000060	0.034630	58.86
VOT	0.87	2418	0.039240	0.011392	0.027849	58.00
VOT	0.89	2821	0.031028	0.016863	0.014166	57.14
VOT	0.89	3224	0.036646	0.011775	0.024872	58.29
VOT	0.88	3627	0.038058	0.010425	0.027633	56.86
VOT	0.90	4030	0.040914	0.014055	0.026859	60.29
VOT	0.89	4030	0.043288	0.013894	0.029395	60.57
VOT	0.88	4433	0.041500	0.015954	0.025546	60.86
VOT	0.91	4836	0.047925	0.005287	0.042638	59.14
STK	0.93	5239	0.046033	0.000194	0.045840	58.29
MEAN	0.87	2907	0.035050	0.007448	0.027603	57.9

Table 7 presents the most comprehensive output analysis of this study and provides a basis for comparison with related work. In this project, we achieved a mean daily ROI of 0.035 and a mean improved ROI of 0.0252 compared to the benchmark, with an overall mean accuracy of 57.9%. As can be seen in Table 7, these values increase with every training iteration, as more data points are accumulated by the algorithm. Therefore, the capability of the second layer, the meta-learner, to make decisions improves over time as more samples are accumulated. Although the meta-learner follows a rule-based approach of selecting the model that had the highest accuracy and precision, the Voting Classifier seems to be the most frequently selected model of instrument-model pairs selection.

Table 8: Comparing different articles

Article	Methodology	Timeframe	Accuracy %	Mean ROI(Daily)
[26]	CHAID	Daily	-	0.001
[25]	Xgboost	Daily	52.23	0.003
[30]	MTDC	10Min	-	0.0044
[29]	TCNN ENSEMBLE	Daily	57.5	0.0117
[21]	GRU	1H	57.08(Precision)	0.025
[20]	MLP	4H	66	0.009-0.03
Ours	CEO	Daily	52.23-60.86	0.035

This paper, in comparison to related works, demonstrates promising results. Additionally, the results presented in Table 8 correspond to an early training phase in our study. Based on Table 7, it is evident that this algorithm can achieve improved performance over time, despite its first layer comprising general classification machine learning models rather than advanced deep learning models. This is due to the application of an ensemble approach that includes a meta-learner layer. Furthermore, Table 8 is ordered based on the mean daily ROI, providing strong evidence that accuracy alone is not a

sufficient metric for evaluating an algorithm. This contrasts with the approach taken in [24], where accuracy was the primary evaluation metric.

5. Conclusion

Results obtained in the experiment testify to the high efficiency of the approach being proposed. Currently, the project has already been tested on real-world data provided by OANDA broker, and computations are performed on the Apple Silicon ARM64 with M2 architecture. This underlines the possibility of obtaining promising results even with managing computational resources. Besides, this project has great potential for possible future improvements. Since the system works like a search engine, constantly looking for an optimal opportunity, it relies heavily on having a flow of data at all times. Further development of the project could be done by increasing the number of financial instruments included, expanding the list of models like CNNs, RNNs, and NLP models [50], [51], or implementing other preprocessing methods into the algorithm, such as advanced oscillators. Improvements might also be made by incorporating more varied and enriched data sources, including financial news, oscillators, reliable model outputs, or correlations between these factors. Though the data and resources are at a premium, the models could still find promising opportunities to lay a foundation for further work.

References

- [1] J. Murphy, *Technical Analysis of the Financial Markets: A Comprehensive Guide to Trading Methods and Applications*, Prentice Hall, 1999.
- [2] A. W. Lo, *Adaptive Markets: Financial Evolution at the Speed of Thought*, Princeton University Press, 2017.
- [3] M. Mathur, S. Mhadalekar, S. Mhatre, V. Mane, Algorithmic trading bot, ITM Web of Conferences 40 (2021) 03041. doi:10.1051/itmconf/20214003041.
- [4] T. Hendershott, C. M. Jones, A. J. Menkveld, Does algorithmic trading improve liquidity?, *The Journal of Finance* 66 (1) (2011) 1–33. doi:10.1111/j.1540-6261.2010.01624.x.
- [5] A. A. Kirilenko, A. W. Lo, Moore’s law versus murphy’s law: Algorithmic trading and its discontents, *Journal of Economic Perspectives* 27 (2) (2013) 51–72. doi:10.1257/jep.27.2.51.
- [6] Á. Cartea, S. Jaimungal, J. Penalva, *Algorithmic and High-Frequency Trading*, Cambridge University Press, 2015.
- [7] D. H. Bailey, J. M. Borwein, M. Lopez de Prado, Q. J. Zhu, Pseudo-mathematics and financial charlatanism: The effects of backtest overfitting on out-of-sample performance, *Notices of the American Mathematical Society* 61 (5) (2014) 458–471. doi:10.1090/noti1105.
- [8] M. Lopez de Prado, *Advances in Financial Machine Learning*, John Wiley & Sons, 2018.
- [9] T. Dietterich, Overfitting and undercomputing in machine learning, *ACM Computing Surveys* 27 (3) (1995) 326–327. doi:10.1145/212094.212114.

- [10] M. Asad, Optimized stock market prediction using ensemble learning, in: 2015 9th International Conference on Application of Information and Communication Technologies (AICT), IEEE, 2015, pp. 263–268. doi:10.1109/ICAICT.2015.7338556.
- [11] R. Saifan, K. Sharif, M. Abu-Ghazaleh, M. Abdel-Majeed, Investigating algorithmic stock market trading using ensemble machine learning methods, *Informatica* 44 (3) (2020) 357–369. doi:10.31449/inf.v44i3.2729.
- [12] C. Krauss, X. A. Do, N. Huck, Deep neural networks, gradient-boosted trees, random forests: Statistical arbitrage on the s&p 500, *European Journal of Operational Research* 259 (2) (2017) 689–702. doi:10.1016/j.ejor.2016.10.031.
- [13] J. X. Wang, Z. Kurth-Nelson, D. Tirumala, H. Soyer, J. Z. Leibo, R. Munos, C. Blundell, D. Kumaran, M. Botvinick, Meta-learning in neural networks: A survey, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44 (9) (2021) 5149–5169. doi:10.1109/TPAMI.2021.3079209.
- [14] M. Lopez de Prado, Building diversified portfolios that outperform out of sample, *The Journal of Portfolio Management* 42 (4) (2016) 59–69. doi:10.3905/jpm.2016.42.4.059.
- [15] G. Naidu, T. Zuva, E. M. Sibanda, A review of evaluation metrics in machine learning algorithms, in: *Computer Science On-line Conference*, Springer, 2023, pp. 15–25. doi:10.1007/978-3-031-34673-5_2.
- [16] D. L. Streiner, G. R. Norman, "precision" and "accuracy": two terms that are neither, *Journal of Clinical Epidemiology* 59 (4) (2006) 327–330. doi:10.1016/j.jclinepi.2005.09.005.
- [17] J. Davis, M. Goadrich, The relationship between Precision-Recall and ROC curves (2006) 233–240doi:10.1145/1143844.1143874.
- [18] K. Olorunnimbe, H. Viktor, Deep learning in the stock market—a systematic survey of practice, backtesting, and applications, *Artificial Intelligence Review* 56 (3) (2023) 2057–2109.
- [19] M. J. Maheronnaghsh, et al., Open peer review on qeios machine learning methods in algorithmic trading: An experimental evaluation of supervised learning techniques for stock price (2023). doi:10.32388/6IJ84P.
- [20] M. Parente, L. Rizzuti, M. Trerotola, A profitable trading algorithm for cryptocurrencies using a neural network model, *Expert Systems with Applications* 238 (2024). doi:10.1016/j.eswa.2023.121806.
- [21] S. Goutte, H.-V. Le, F. Liu, H.-J. Von Mettenheim, Deep learning and technical analysis in cryptocurrency market, *Finance Research Letters* 54 (2023) 103809.
- [22] M. T. Rahman, R. Akhter, S. A. Mazumder, K. Lawal, M. A. Rahman, Forecasting and pattern analysis of dhaka stock market using lstm and prophet algorithm, Preprint (2021). doi:10.13140/RG.2.2.18906.56001.

- [23] H. S. Ringmu, S. B. Oumar, H. S. Ringmu, Saidou, B. Oumar, Forecasting stock prices in the new york stock exchange, *Journal of Economic and Business* 9 (1) (2022). doi:10.1453/jeb.v9i1.2269.
- [24] M. Liang, S. Wu, X. Wang, Q. Chen, A stock time series forecasting approach incorporating candlestick patterns and sequence similarity, *Expert Systems with Applications* 205 (2022). doi:10.1016/j.eswa.2022.117595.
- [25] Y. Santur, Candlestick chart based trading system using ensemble learning for financial assets, *Sigma Journal of Engineering and Natural Sciences* 40 (2) (2022) 370–379.
- [26] S. Thammakesorn, O. Sornil, Generating trading strategies based on candlestick chart pattern characteristics, in: *Journal of Physics: Conference Series*, Vol. 1195, IOP Publishing, 2019, p. 012008.
- [27] S. Nison, *Japanese Candlestick Charting Techniques*, New York Institute of Finance, 2001.
- [28] T. G. Dietterich, Ensemble methods in machine learning, in: *International workshop on multiple classifier systems*, Springer, 2000, pp. 1–15.
- [29] S. Ghoshal, S. Roberts, Thresholded convnet ensembles: neural networks for technical forecasting, *Neural Computing and Applications* 32 (18) (2020) 15249–15262.
- [30] A. Adegboye, M. Kampouridis, F. Otero, Algorithmic trading with directional changes, *Artificial Intelligence Review* 56 (6) (2023) 5619–5644. doi:10.1007/s10462-022-10307-0.
- [31] A. Hagmann, *Algorithmic trading with python and machine learning* (2023). URL <https://github.com/hagmann/algorithmic-trading-with-python-and-machine-learning>.
- [32] R. Cooper, M. Ong, B. Van Vliet, Multi-scale capability: A better approach to performance measurement for algorithmic trading, *Algorithmic Finance* 4 (1-2) (2015) 53–68. doi:10.3233/AF-150043.
- [33] R. J. Hyndman, *Moving averages* (2009). URL <https://cran.r-project.org/web/packages/forecast/vignettes/ma.pdf>
- [34] A. Țăran Moroșan, The relative strength index revisited, *African Journal of Business Management* 5 (14) (2011) 5855–5862. URL <http://www.academicjournals.org/ajbm>
- [35] Z. H. Lashkary, *Technical analysis of forex by macd indicator* (2015). URL <https://www.researchgate.net/publication/274071851>
- [36] W. E. Marcílio, D. M. Eler, From explanations to feature selection: assessing shap values as feature selection mechanism, in: *2020 33rd SIBGRAPI conference on Graphics, Patterns and Images (SIBGRAPI)*, Ieee, 2020, pp. 340–347.

- [37] Y. Liu, Z. Li, R. Nekhili, J. Sultan, Forecasting cryptocurrency returns with machine learning, *Research in International Business and Finance* 64 (2023). doi:10.1016/j.ribaf.2023.101905.
- [38] D. Ruta, B. Gabrys, Classifier selection for majority voting, *Information fusion* 6 (1) (2005) 63–81.
- [39] S. Džeroski, B. Ženko, Is combining classifiers with stacking better than selecting the best one?, *Machine learning* 54 (2004) 255–273.
- [40] J. H. Friedman, Greedy function approximation: a gradient boosting machine, *Annals of statistics* (2001) 1189–1232.
- [41] D. W. Hosmer Jr, S. Lemeshow, R. X. Sturdivant, *Applied logistic regression*, John Wiley & Sons, 2013.
- [42] S. R. Safavian, D. Landgrebe, A survey of decision tree classifier methodology, *IEEE transactions on systems, man, and cybernetics* 21 (3) (1991) 660–674.
- [43] L. Breiman, Random forests, *Machine learning* 45 (2001) 5–32.
- [44] J. M. Keller, M. R. Gray, J. A. Givens, A fuzzy k-nearest neighbor algorithm, *IEEE transactions on systems, man, and cybernetics* (4) (1985) 580–585.
- [45] K. P. Murphy, et al., Naive bayes classifiers, *University of British Columbia* 18 (60) (2006) 1–8.
- [46] L. Ladickỳ, P. H. Torr, Locally linear support vector machines (2011).
- [47] H. Taud, J.-F. Mas, Multilayer perceptron (mlp), in: *Geomatic approaches for modeling land change scenarios*, Springer, 2017, pp. 451–455.
- [48] R. G. Brereton, G. R. Lloyd, Support vector machines for classification and regression, *Analyst* 135 (2) (2010) 230–267.
- [49] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al., Scikit-learn: Machine learning in python, the *Journal of machine Learning research* 12 (2011) 2825–2830.
- [50] R. K. Karanam, et al., Neural networks in algorithmic trading for financial markets, *Asian Accounting and Auditing Advancement* 9 (1) (2018).
- [51] S. from Fervent, Investment analysis with natural language processing nlp (2023).